

SEVENTH EDITION

Systems Analysis AND Design

IN A CHANGING WORLD

Chapter 12 & 13

JOHN SATZINGER

ROBERT JACKSON

STEPHEN BURD

COMP-CO835

Object Oriented Systems

Week 11 Lecture

Professor: Dr. Stanley Ukah

Outline

- Object-Oriented Design: Bridging from Analysis to Implementation
- Steps of Object-Oriented Design
- Design Classes and the Design Class Diagram
- Designing with CRC Cards
- Fundamental Principles for Good Design

Outline

- Object-Oriented Design with Interaction Diagrams
- Use Case Realization with Communication Diagrams
- Use Case Realization with Sequence Diagrams
- Developing a Multilayer Design
- Updating and Packaging the Design Classes
- Design Patterns

Learning Objectives

- Explain the purpose and objectives of object-oriented design
- Develop design class diagrams
- Use C R C cards to define class responsibilities and collaborations
- Explain the different types of objects and layers in a design
- Develop communication diagrams for use case realization
- Develop sequence diagrams for use case realization
- Explain design patterns and recognize various specific patterns

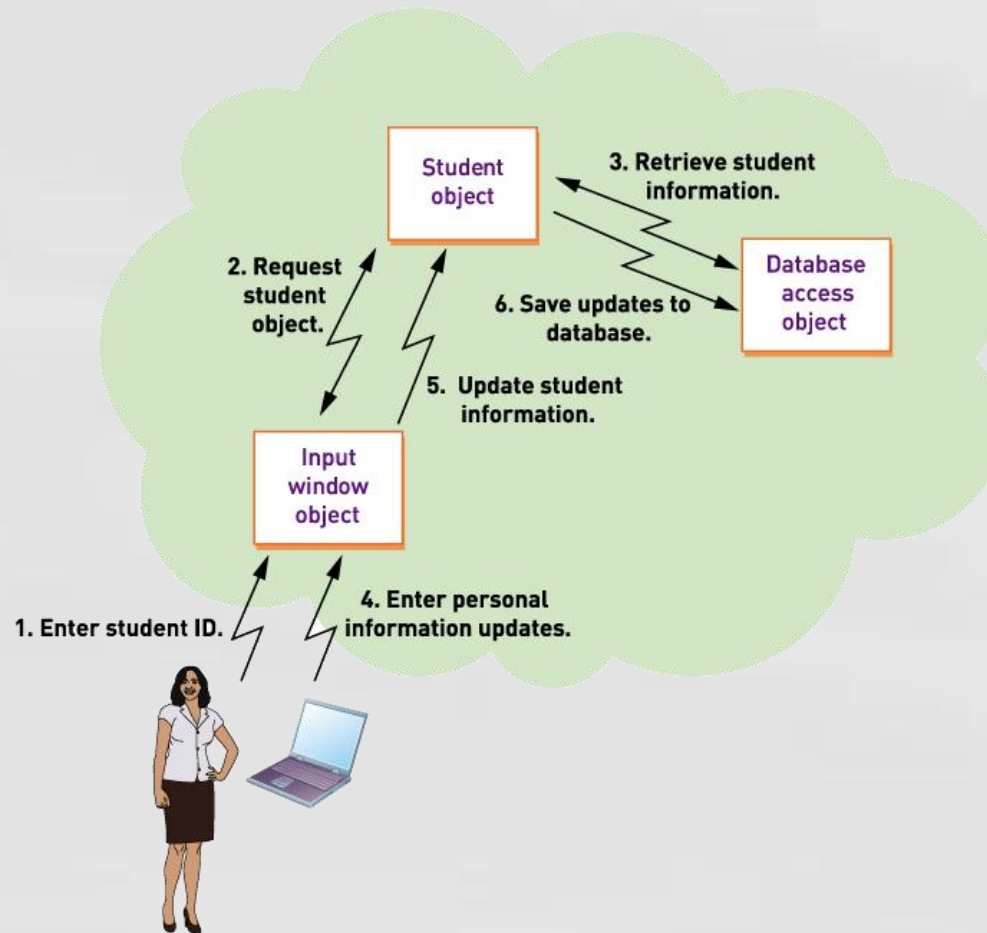
Overview

- These two chapters (12&13) focus on designing software for the new system, at both the architectural and detailed level design
- Design models are based on the requirements models learned in Chapters 3, 4, and 5
- The steps of object-oriented design are explained
- The main model discussed is the design class diagram
- The C R C Cards technique is used to design the OO software
- Three layer design is demonstrated using communication diagrams, sequence diagrams, package diagrams, and design patterns
- Design is shown to proceed use case by use case, and within each use case, layer by layer

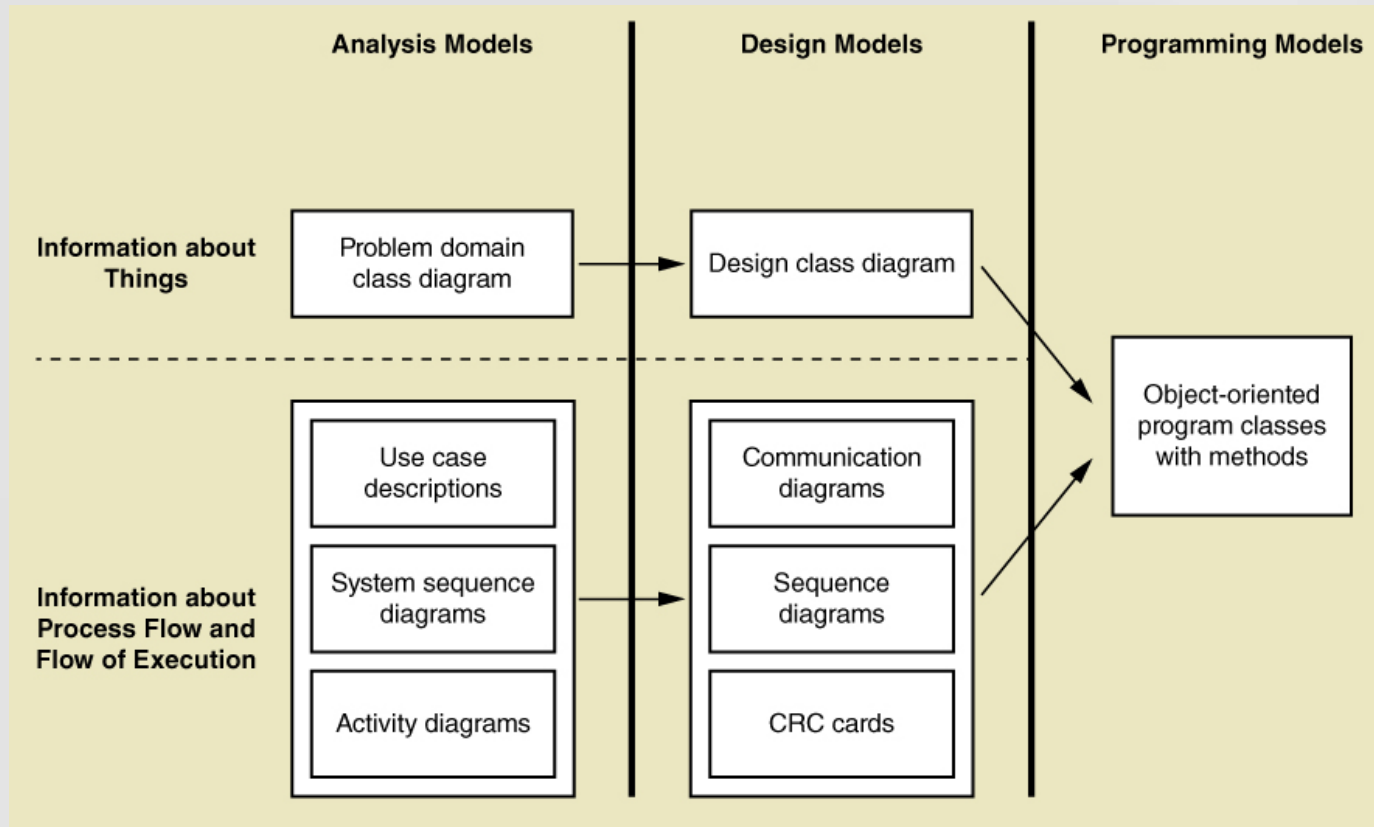
OO Design: Bridging from Analysis to Implementation

- OO Design: Process by which a set of detailed OO design models are built to be used for coding
- Strength of OO is requirements models from Chapters 3, 4, and 5 are extended to design models. No reinventing the wheel
- Design models are created in parallel to actual coding/implementation with iterative SDLC
- Agile approach says create models only if they are necessary. Simple detailed aspects don't need a design model before coding

Object-Oriented Program Flow: Three Layer Architecture



Analysis to Design to Implementation: Model Flow



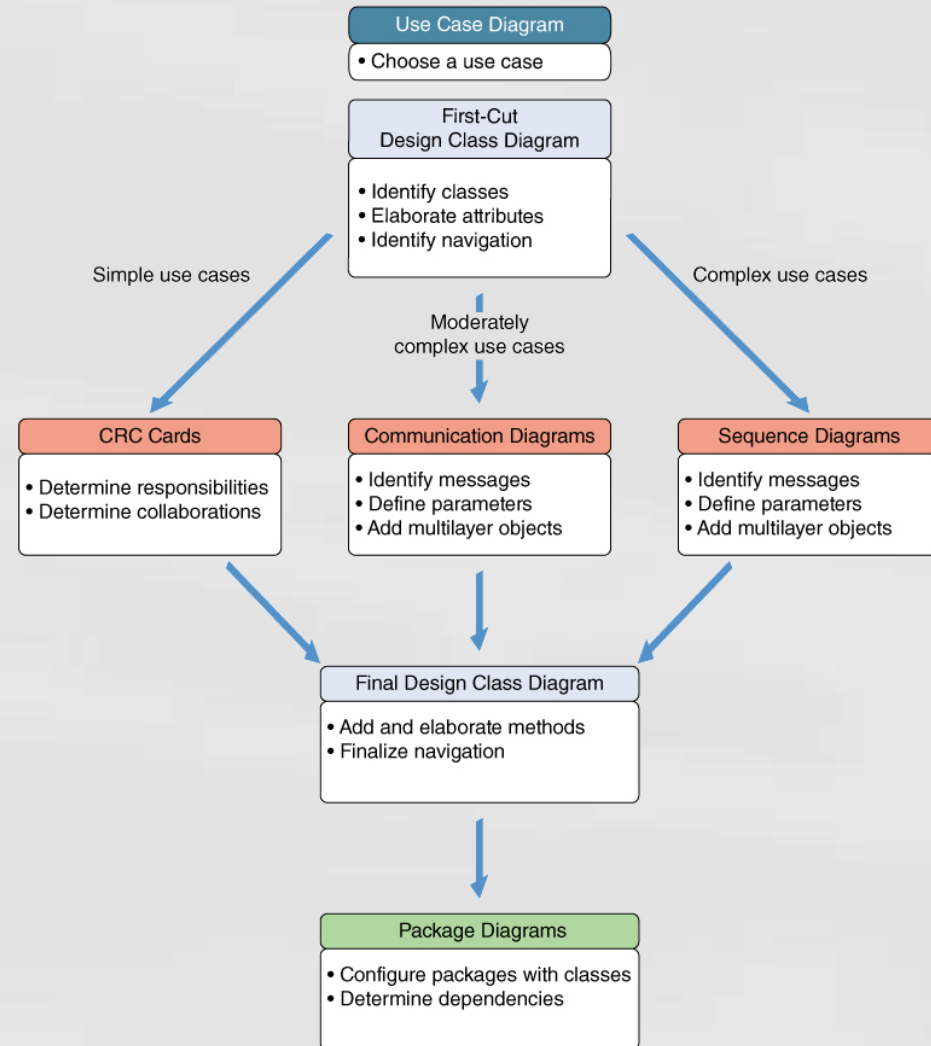
Steps of Object-Oriented Design (1 of 2)

- Object-oriented design
 - The process to identify the classes, their methods and the messages required for a use case
- Use case driven
 - Design is carried out use case by use case

Steps of Object-Oriented Design (2 of 2)

- Three paths

- Simple use case use CRC Cards
- Medium use case use Communication Diagram
- Complex use case use Sequence Diagram



Design Class Diagrams

- **stereotype** a way of categorizing a model element by its characteristics, indicated by guillemots (<< >>)
- **persistent class** a class whose objects exist after a system is shut down (data remembered)
- **entity class** a design identifier for a problem domain class (usually persistent)
- **boundary class or view class** a class that exists on a system's automation boundary, such as an input window form or Web page
- **controller class** a class that mediates between boundary classes and entity classes, acting as a switchboard between the view layer and domain layer
- **data access class** a class that is used to retrieve data from and send data to a database

First-Cut Design Class Diagram

- Proceed use case by use case, adding to the diagram
- Pick the domain classes that are involved in the use case (see preconditions and post conditions for ideas)
- Add a controller class to be in charge of the use case
- Determine the initial navigation visibility requirements using the guidelines and add to diagram
- Elaborate the attributes of each class with visibility and type
- Note that often the associations and multiplicity are removed from the design class diagram as in text to emphasize navigation, but they are often left on

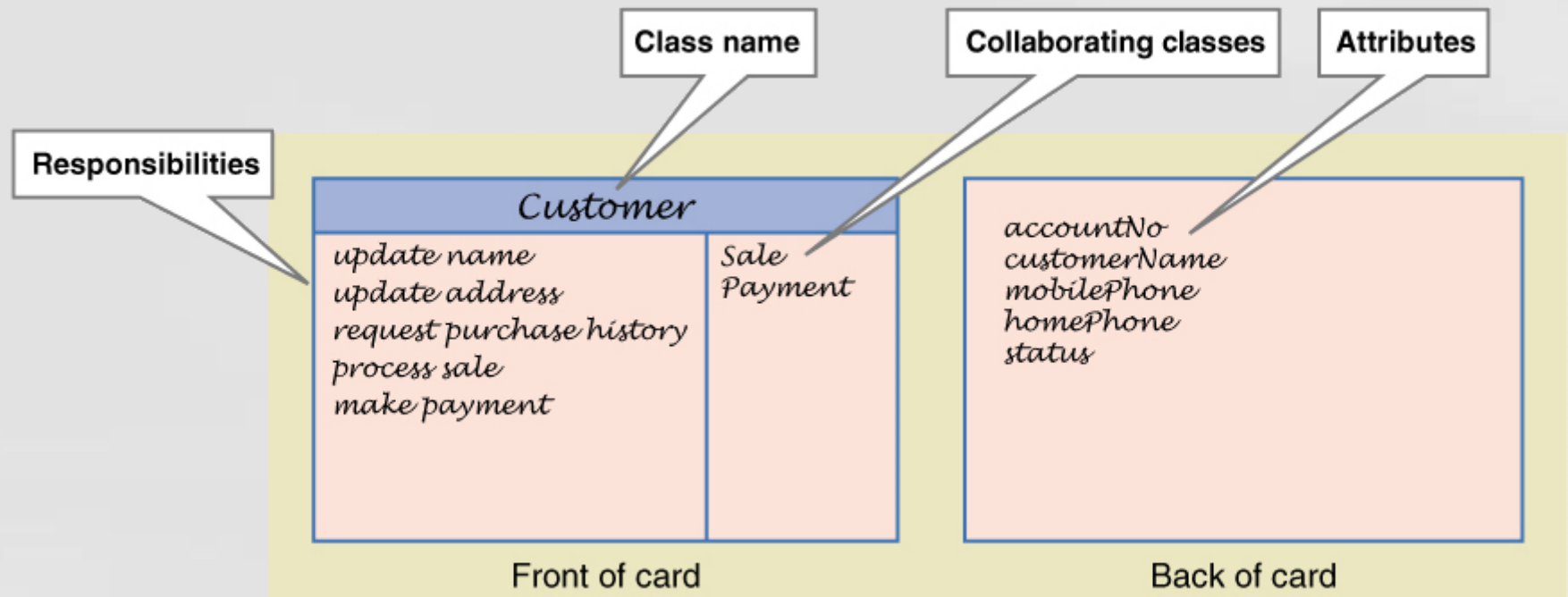
Designing With CRC Cards

- CRC Cards—Classes, Responsibilities, Collaboration Cards
- OO design is about assigning Responsibilities to Classes for how they Collaborate to accomplish a use case
- Usually a manual process done in a brainstorming session
 - 3 × 5 note cards
 - One card per class
 - Front has responsibilities and collaborations
 - Back has attributes needed

CRC Cards

- CRC cards help developers and designers think through how different classes in a system interact with each other, defining the key responsibilities of each class and identifying their collaborators.
 - . **Classes:** Represent key components or objects in a system.
 - . **Responsibilities:** Indicate the duties or functions the class must perform.
 - . **Collaborators:** Show the other classes that the given class interacts with to fulfill its responsibilities.

Example of CRC Card



Fundamental Design Principles (1 of 6)

● Object Responsibility

- A design principle that states objects are responsible for carrying out system processing
- A fundamental assumption of OO design and programming
- Responsibilities include “knowing” and “doing”
- Objects know about other objects (associations) and they know about their attribute values. Objects know how to carry out methods, do what they are asked to do.
- Note that CRC cards and the design in the next chapter involve assigning responsibilities to classes to carry out a use case.
- If deciding between two alternative designs, choose the one where objects are assigned responsibilities to collaborate to complete tasks (don't think procedurally).

Fundamental Design Principles (2 of 6)

- Separation of Responsibilities
 - AKA Separation of Concerns
 - Applied to a group of classes
 - Segregate classes into packages or groups based on primary focus of the classes
 - Basis for multi-layer design – view, domain, data
 - Facilitates multi-tier computer configuration

Fundamental Design Principles (3 of 6)

● Protection from Variations

- A design principle that states parts of a system unlikely to change are separated (protected) from those that will surely change
- Separate user interface forms and pages that are likely to change from application logic
- Put database connection and SQL logic that is likely to change in a separate classes from application logic
- Use adaptor classes that are likely to change when interfacing with other systems
- If deciding between two alternative designs, choose the one where there is protection from variations

Fundamental Design Principles (4 of 6)

● Indirection

- A design principle that states an intermediate class is placed between two classes to decouple them but still link them
- A controller class between UI classes and problem domain classes is an example
- Supports low coupling
- Indirection is used to support security by directing messages to an intermediate class as in a firewall
- If deciding between two alternative designs, choose the one where indirection reduces coupling or provides greater security

Fundamental Design Principles (5 of 6)

● Coupling

- A quantitative measure of how closely related classes are linked (tightly or loosely coupled)
- Two classes are tightly coupled if there are lots of associations with another class
- Two classes are tightly coupled if there are lots of messages to another class
- It is best to have classes that are **loosely coupled**
- If deciding between two alternative designs, choose the one where overall coupling is less

Fundamental Design Principles (6 of 6)

● Cohesion

- A quantitative measure of the focus or unity of purpose within a single class (high or low cohesiveness)
- One class has high cohesiveness if all of its responsibilities are consistent and make sense for purpose of the class (a customer carries out responsibilities that naturally apply to customers)
- One class has low cohesiveness if its responsibilities are broad or makeshift
- It is best to have classes that are **highly cohesive**
- If deciding between two alternative designs, choose the one where overall cohesiveness is high

OOD with Interaction Diagrams

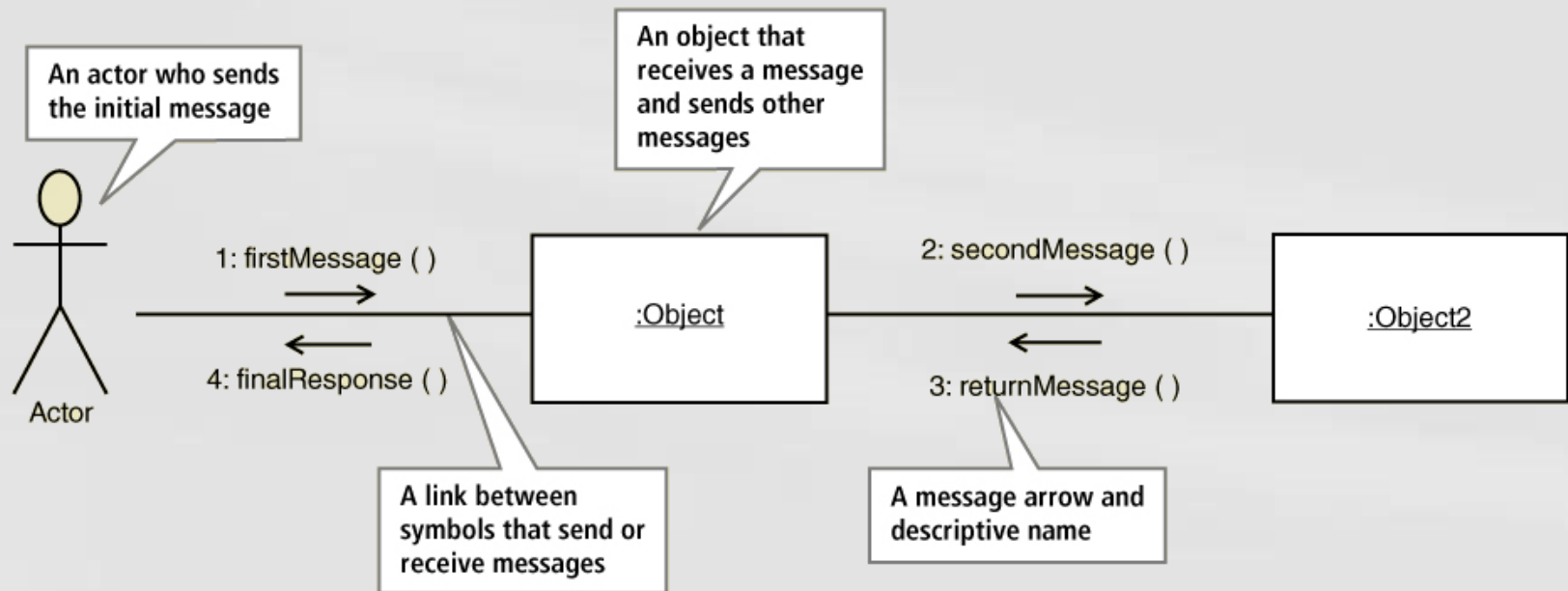
- Use case realization
 - The process of elaborating the detailed design for a particular use case using interaction diagrams
- Communication diagram
 - A type of interaction diagram which emphasizes the set of objects involved in a use case
- Sequence diagram
 - A type of interaction diagram which emphasizes the sequence of messages involved in a use case

Understanding Communication Diagrams

- Actor – the external role of the person or thing that initiates the use case. Sends messages.
- Object – the instantiated class objects that perform the actions (methods) to execute the use case. They receive messages and process messages.
- Link – simply connectors between objects to carry the messages.
- Message – the requests for service with an originating actor or object and a destination object, which performs the requested service

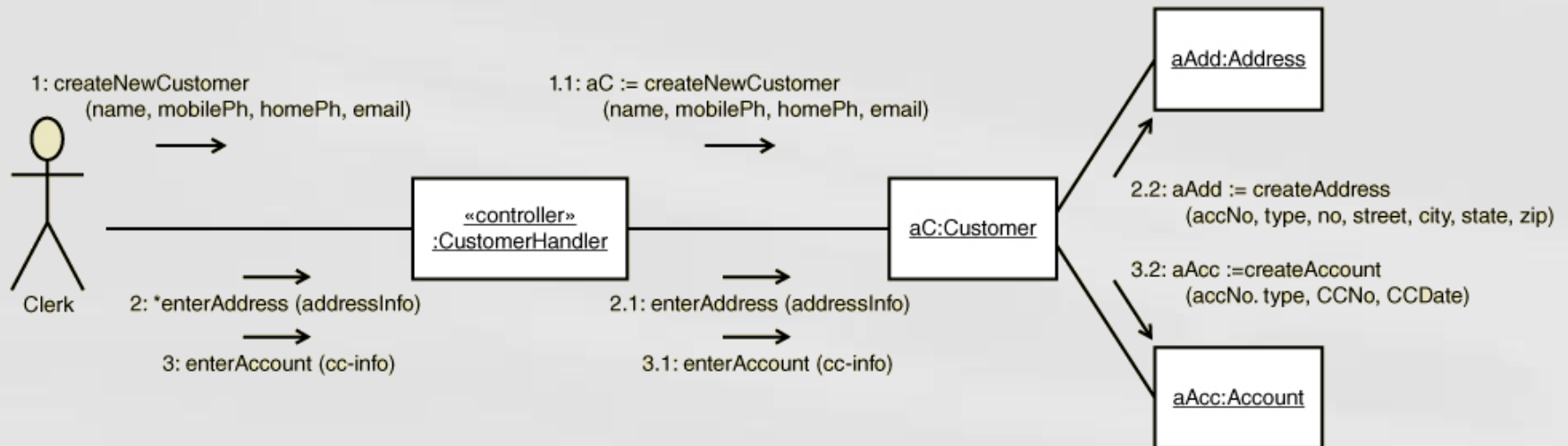
Understanding Communication Diagrams

- Symbols used in a communication diagram:



Example Communication Diagram:

● *Create customer account* use case

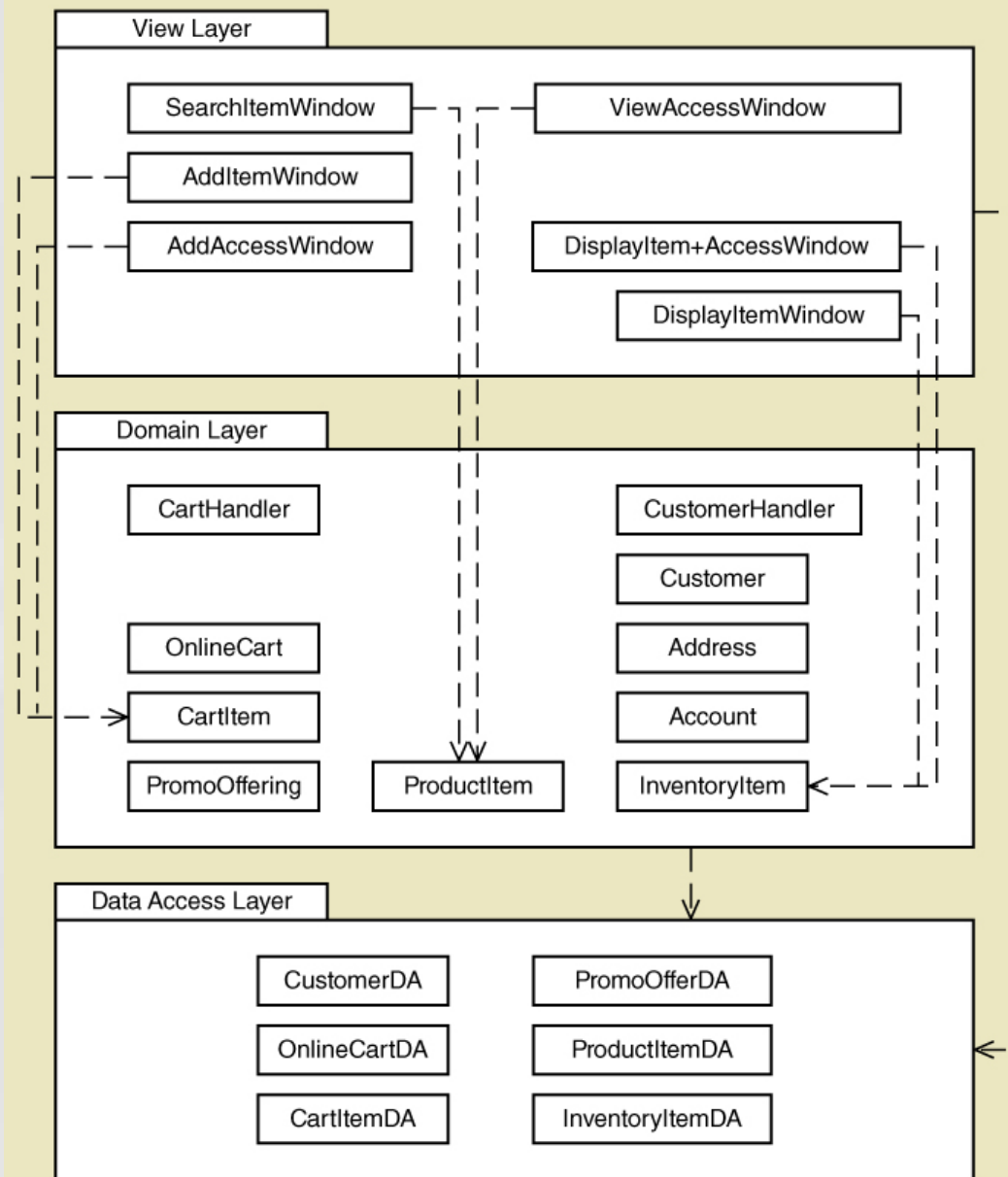


Package Diagrams

- Can be used to define formal packages such as subsystems
- Can be used informally to group classes together for understanding
- Dependency relationship
 - A relationship between packages or classes within a package in which a change of the independent component may require a change in the dependent component. Indicated by dashed line. Read in the direction of the arrow, i.e. $A \rightarrow B$ is A depends on B.

Package Diagram

- Three-layer package diagram of classes in Figure 13-24
- Dependencies
 - View layer depends on Data Access layer
 - SearchItemWindow depends on ProductItem
 - etc.



Implementation Issues

Three Layer Design

- View Layer Class Responsibilities
 - Display electronic forms and reports.
 - Capture such input events as clicks, rollovers, and key entries.
 - Display data fields.
 - Accept input data.
 - Edit and validate input data.
 - Forward input data to the domain layer classes.
 - Start and shut down the system.

Implementation Issues

Three Layer Design

- Domain Layer Class Responsibilities
 - Create problem domain (persistent) classes.
 - Process all business rules with appropriate logic.
 - Prepare persistent classes for storage to the database.
- Data Access Layer Class Responsibilities
 - Establish and maintain connections to the database.
 - Contain all SQL statements.
 - Process result sets (the results of SQL executions) into appropriate domain objects.
 - Disconnect gracefully from the database.

Design Patterns

- Design patterns became widely accepted after the publication of *Elements of Reusable Object-Oriented Software*, (1996) by Gamma et al (the “Gang of Four”)
- **A design pattern is a reusable solution to a common problem in system design.**
- There are architectural design patterns talked about already
 - Three layer or model-view-controller architecture
- The first example of a programming design pattern shown is the Controller Pattern.
 - Problem is deciding how to handle all of the messages from the view layer to classes in the problem domain layer to reduce coupling
 - Solution is to assign one class between the view layer and the problem domain layer that receives all messages and acts as a switchboard directing messages to the problem domain

Design Patterns

- Design Pattern—standard design techniques and templates that are widely recognized as good practice
- For common design/coding problems, the design pattern suggests the best way to handle the problem.
- They are written up in design pattern catalogs/references. Include:
 - Pattern name
 - Problem that requires solution
 - The pattern that solves the problem
 - An example of the pattern
 - Benefits and consequences of the a pattern

Controller Pattern

First step toward multilayer architecture

The **Controller Design Pattern** acts as an intermediary between the **view** (user interface) and the **model** (data/business logic). It handles user input, processes it by interacting with the model, and updates the view accordingly.

Name:	Controller
Problem:	Domain classes have the responsibility of processing use cases. However, since there can be many domain classes, which one(s) should be responsible for receiving the input messages? User-interface classes become very complex if they have visibility to all of the domain classes. How can the coupling between the user-interface classes and the domain classes be reduced?
Solution:	Assign the responsibility for receiving input messages to a class that receives all input messages and acts as a switchboard to forward them to the correct domain class. There are several ways to implement this solution: (a) Have a single class that represents the entire system, or (b) Have a class for each use case or related group of use cases to act as a use case handler.
Example:	The RMO Customer account subsystem accepts inputs from a <code>:CustomerForm</code> window. These input messages are passed to the <code>:CustomerHandler</code>, which acts as the switchboard to forward the message to the correct problem domain class. <pre> graph LR subgraph UI [User interface] RMO[RMO New Customer] CF[:CustomerForm] end subgraph DC [Domain classes] CH[:CustomerHandler] C[:Customer] end RMO -- "createNewCustomer()" --> CF CF -- "createNewCustomer()" --> CH CH -- "createNewCustomer()" --> C style CH stroke-dasharray: 5 5 </pre> Other cases of the controller pattern will be used for each RMO use case.
Benefits and consequences:	Coupling between the view layer and the domain layer is reduced. The controller provides a layer of indirection. The controller is closely coupled to many domain classes. If care is not taken, controller classes can become incoherent, with too many unrelated functions. If care is not taken, business logic will be inserted into the controller class.

More Design Patterns

- Adapter
 - Like an electrical adapter
 - Place an adapter class between your system and an external system
- Factory
 - Use factor class when creation logic is complex for a set of classes
- Singleton
 - Use when only one instance should exist at a time and is shared

Adapter Design Pattern

- The Adapter pattern allows incompatible interfaces to work together by providing a wrapper that translates calls/requests from one interface to another.

Name:	Adapter
Problem:	A class must be replaced, or is subject to being replaced, by another standard or purchased class. The replacing class already has a predefined set of method signatures that are different from the method signatures of the original class. How do you link in the new class with a minimum of impact so that you don't have to change the names throughout the system to the method names in the new class?
Solution:	Write a new class, the adapter class, which serves as a link between the original system and the class to be replaced. This class has method signatures that are the same as those of the original class (and the same as those expected by the system). Each method then calls the correct desired method in the replacement class with the method signature. In essence, it "adapts" the replacement class so that it looks like the original class.
Example:	There are several places in the RMO system where class libraries were purchased to provide special processing. These purchased libraries provide specialized services such as tax calculations and shipping and postage rates. From time to time, these service libraries are updated with new versions. Sometimes a service library is even replaced with one from an entirely different vendor. The RMO systems staff applies protection from variations and indirection design principles by placing an adapter in front of each replaceable class. <pre> classDiagram class System class TaxCalculatorIF { <<interface>> getSTax() getUTax() } class TaxCalcAdapter { getSTax() getUTax() } class ABCTaxCalculator { findTax1() findTax2() } System --> TaxCalculatorIF TaxCalcAdapter .. > TaxCalculatorIF TaxCalcAdapter --> ABCTaxCalculator </pre>
Benefits and consequences:	The adaptee class can be replaced as desired. Changes are confined to the adapter class and do not ripple through the system. Two classes are defined, an interface class and the adapter class. Passed parameters may add more complexity, and it is difficult to limit changes to the adapter class.

Factory Design Pattern

The Factory Method pattern defines an interface for creating an object but lets subclasses alter the type of objects that will be created.

Example: A document creator application that supports multiple formats (PDF, Word, etc.).

Name:	Factory or Factory Method
Problem:	Who should be responsible for creating utility type objects that do not specifically belong to the problem domain classes? These utility objects may also be accessed from various places within the system, so a given object may need to be instantiated from several classes.
Solution:	Create an artifact that is a factory class. Its responsibility is only to instantiate utility classes. In many situations, only one instance of a particular utility class is allowed. Hence, all classes that need access to the class come through the factory. The factory ensures that only one instance is created.
Example:	Several places in the RMO system need to get data from an Order object and need to have a reference to an Order_DA [data access] object. The Order_DA object may or may not already have been instantiated. A data access factory is defined and an interface is created. The requesting object uses the methods defined in the interface to request the reference to the Order_DA object. It then can read the database of orders. <pre> classDiagram class System { Customer Order Shipment } class Order_DA class DAFactory_iF { <<interface>> getOrder_DA() getCustomer_DA() getShipment_DA() } class DAFactory { -myODA: Order_DA -myCDA: Customer_DA -mySAD: Shipment_DA getOrder_DA() getCustomer_DA() getShipment_DA() } System --> Order_DA : «uses» Order --> Order_DA : «requiresDA» Order --> DAFactory_iF : «requiresDA» DAFactory_iF < -- DAFactory DAFactory --> Order_DA : «creates» </pre> <pre> public synchronized Order_DA getOrder_DA () { if (myODA == null) { myODA = new Order_DA (); } return myODA; } </pre>
Benefits and Consequences:	Higher cohesion of problem domain classes Less coupling between business logic layer and data layer Smaller, more maintainable classes

Singleton Design Pattern

- The Singleton pattern ensures that a class has only one instance and provides a global point of access to that instance.
- This is useful when exactly one object is needed to coordinate actions across the system.

Name:	Singleton
Problem:	Only one instantiation of a class is allowed. The instantiation (new) can be called from several places in the system. The first reference should make a new instance, and later attempts should return a reference to the already instantiated object. How do you define a class so that only one instance is ever created?
Solution:	A singleton class has a static variable that refers to the one instance of itself. All constructors to the class are private and are accessed through a method or methods, such as getInstance(). The getInstance() method checks the variable; if it is null, the constructor is called. If it is not null, then only the reference to the object is returned.
Example:	In RMO's system, the connection to the database is made through a class called Connection. However, for efficiency, we want each desktop system to open and connect to the database only once, and to do so as late as possible. Only one instance of Connection—that is, only one connection to the database—is desired. The Connection class is coded as a singleton. The following coding example is similar to C# and Java: <pre> Class Connection { private static Connection conn = null; public synchronized static getConnection () { if (conn == null) { conn = new Connection (); } return conn; } </pre> Another example of a singleton pattern is a utilities class that provides services for the system, such as a factory pattern. Because the services are for the entire system, it causes confusion if multiple classes provide the same services. An additional example might be a class that plays audio clips. Since only one audio clip should be played at one time, the audio clip manager will control that. However, for this to work, there must be only one instance of the audio clip manager.
Benefits and consequences:	There are other times when only one instance of an object is needed, but if it is instantiated from only one place, then a singleton may not be required. The singleton object controls itself and ensures that only one instance is created—no matter how many times it is called and wherever the call occurs in the system. The code to implement the singleton is very simple, which is one of the desirable characteristics of a good design pattern.

In-Class Task

- You are tasked with designing a library management system that handles book checkouts, returns, and member records. The system should include book inventory, member management, and transaction records.
 - Brainstorm and identify potential classes such as: Book, Member, LibraryCatalog, Transaction, and LibraryStaff,
 - Each group should create CRC cards for each identified class.
- List 2-3 primary responsibilities for each class.
 - Example for Book: Maintain book information, check book availability, and update book status.
- List which other classes each class needs to collaborate with.
 - Example for Transaction: Collaborates with Book for checkout and Member for borrowing history.
- Each group will present their CRC cards and explain their design choices to the class. Provide a short explanation of the rationale behind your chosen responsibilities and collaborations.

Example CRC Card:

Class Name	Book
Responsibilities	- Maintain book details
	- Check availability
	- Update book status
Collaborators	LibraryCatalog , Transaction

Summary (1 of 3)

- System Design solves business problems by bridging the gap between analysis and implementation.
- System Design proceeds use case by use case, sometimes called “use case driven” and the design of each use case is called use case realization.
- The process of design proceeds along three paths depending on the complexity of the use case. Simple use cases use CRC cards, medium complexity uses communication diagrams, complex use cases proceed with sequence diagrams.

Summary (2 of 3)

- Design class diagrams include additional notation because design classes are now software classes.
- CRC (Class-Responsibility-Collaboration) cards are a simple, effective tool used in object-oriented design to model and understand the structure of a system.
- CRC cards help developers and designers think through how different classes in a system interact with each other, defining the key responsibilities of each class and identifying their collaborators.
- A design pattern is a reusable solution to a common problem in system design.

Summary (3 of 3)

- C R C Cards technique can be used to design how the classes collaborate to complete each use case. C R C stands for Classes, Responsibilities, and Collaborations.
- Decisions about design options are guided by some fundamental design principles. Some of these are coupling, cohesion, protection from variations, indirection, and object responsibility.
- The UML package diagram is used to structure the classes into packages, usually one package per layer. The package diagram can also be used to package layers into subsystems.
- The view layer can also be added to the sequence diagram to show how multiple pages or forms interact with the use case controller.

Questions?